BrainScope — Intermediate progress report

Alejandro Bordon — Medical Image Processing (11763)

May 4, 2026

Repository. `https://github.com/AlejandroB10/BrainScope` (public, contains the project skeleton and the planning artifacts referenced in this report).

1. Project overview

BrainScope is the implementation of the project proposal as a Python pipeline. It works on a single co-acquired study formed by the dynamic PET acquisition `e_1_BRAIN_DINAMIC_COLINA` and the MR T1 reference `AX_3D_T1`, and it follows the three objectives of the proposal: DICOM loading and visualization, 3D rigid coregistration of the temporally averaged PET onto the MR reference, and semi-automatic segmentation of the tumor visible in the last PET frame using an AI general-purpose model.

2. Current state

This intermediate milestone is deliberately about planning and infrastructure rather than running code. The repository is public and follows a clean Python layout (`src/`, `data/`, `docs/`, `openspec/`, `results/`) with a conda environment file, a `.gitignore` tuned for DICOM data and generated artifacts, and module stubs for each objective. The conda environment `medical-image-processing-11763` (Python 3.11, with `pydicom`, `numpy`, `scipy`, `matplotlib`, `scikit-image` and `SimpleITK`) is the one used across the course activities. Both DICOM studies are already downloaded locally and the relevant headers are catalogued in `data/README.md`. 3D Slicer 5.10 is installed for qualitative inspection, and the `mcp-slicer` bridge is being configured so the visual checks can be driven from the development workflow. Planning is tracked through openspec: the change `intermediate-submission` keeps a proposal, a design note, and a tasks checklist under `openspec/changes/`, and future objectives will follow the same template.

3. Planned approach per objective

Objective 1 — DICOM loading and visualization. The dynamic PET `pixel_array` will be reshaped into a 4D volume `(frame, slice, row, col)` using `Number of Frames`, `Frame Positions Vector`, `Frame Start Times Vector`, and `Frame Durations`. Spacing is recovered from `Pixel Spacing` and `Spacing Between Slices`. Static visualizations include the last frame and the temporal mean. A GIF will sweep the three median planes (axial, coronal, sagittal) across all temporal frames, reusing the `MIP_*`, `AIP_*`, and rotating projection helpers built in `activity03`.

Objective 2 — 3D rigid coregistration. The temporally averaged PET volume will be registered to the MR T1 volume with a rigid transform (translation plus axial rotation), parameterized exactly as in `activity06`. Initial parameters align both centroids. The optimizer is `scipy.optimize.least_squares`, driving a Mutual Information loss computed from a 32-bin joint histogram (the `mutual_information` helper of `activity07` is the starting point). PyElastix is the candidate fallback if the home-made pipeline becomes unstable. A rotating MIP animation on the coronal-sagittal planes will overlay the reference, the coregistered input, and an alpha-fused composite of both.

Objective 3 — 3D image segmentation. The tumor visible in the last PET frame is located manually (centroid + bounding box) and the prompt is fed to a general-purpose AI model that returns a 3D mask on the MR volume. MedSAM2 is the current candidate because it propagates masks natively in 3D and has a medical fine-tune; nnInteractive, SAMed-2, and SAT are still on the table. The final mask is assessed visually against the coregistered

PET and quantified through volume and shape descriptors, plus overlap metrics (Dice, Jaccard) if a reference mask is provided.

4. Risks and mitigations

Risk	Mitigation
Dynamic PET frame ordering misinterpreted from the headers	Cross-check the reshaped volume against 3D Slicer's native player
Rigid coregistration converges to a local minimum	Compare home-made result against PyElastix; use centroid-based init
AI segmentation model not directly applicable to PET-only tumors	Apply the model on the MR volume after coregistration, then back-project
Time pressure for the AI segmentation pipeline	Tackle Objectives 1 and 2 first to unlock most of the grade early

5. Next steps

The next change opened in `openspec/` is `objective-1-dicom-loading`. After that, the implementation proceeds in the order given by the proposal. The companion document `questions.md` collects the open decisions that would benefit from feedback before the implementation starts.